

Python Programming

Day 9: Matplotlib 기초

Line Plot, Styling, Multiple Plots, Bar, Scatter, Histogram, Subplot

Contents

1	학습 목표	2
2	Matplotlib와 기본 선 그래프	2
2.1	기본 선 그래프	2
2.2	최종 코드: 01_basic_plot.py	2
3	그래프 스타일	3
3.1	최종 코드: 02_plot_style.py	3
4	여러 개의 선 그래프	4
4.1	최종 코드: 03_multiple_plots.py	4
5	막대 그래프	4
5.1	최종 코드: 04_bar_chart.py	4
6	산점도	5
6.1	최종 코드: 05_scatter_plot.py	5
7	히스토그램	6
7.1	최종 코드: 06_histogram.py	6
8	여러 그래프 배치	6
8.1	최종 코드: 07_subplot.py	7
9	Day 9 종합 실습: 학생 데이터 시각화	7
9.1	프로그램 요구사항	7
9.2	최종 코드: practice_day09.py	8
9.3	프로그램 구조 분석	9
10	실습 파일 목록	9
11	핵심 요약	9

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- Matplotlib의 목적과 pyplot의 기본 사용법 설명하기
- plot()으로 기본 선 그래프 그리기
- 색상, 선 스타일, 마커, 범례를 이용해 그래프 꾸미기
- 하나의 좌표축에 여러 개의 선 그래프 그리기
- bar()로 막대 그래프 그리기
- scatter()로 두 변수의 관계 나타내기
- hist()로 데이터의 분포 확인하기
- subplot()과 tight_layout()으로 여러 그래프 배치하기

2. Matplotlib와 기본 선 그래프

Matplotlib는 Python에서 데이터를 그래프로 시각화하기 위한 대표적인 라이브러리이다. 수치 데이터의 변화, 항목별 비교, 변수 간 관계, 데이터 분포 등을 다양한 형태의 그래프로 표현할 수 있다.

Matplotlib를 설치하지 않았다면 터미널에서 다음 명령을 한 번 실행한다.

```
pip install matplotlib
```

그래프를 그릴 때는 일반적으로 matplotlib.pyplot을 plt라는 별칭으로 불러온다.

```
1 import matplotlib.pyplot as plt
```

2.1. 기본 선 그래프

plt.plot(x, y)는 x값과 y값을 이용해 각 데이터 점을 연결한 선 그래프를 만든다. 그래프를 실제 화면에 표시하려면 마지막에 plt.show()를 호출한다.

제목과 축 이름은 각각 title(), xlabel(), ylabel()로 지정할 수 있으며, grid(True)를 사용하면 격자를 표시할 수 있다.

2.2. 최종 코드: 01_basic_plot.py

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 4, 5]
4 y = [2, 4, 6, 8, 10]
5
6 plt.plot(x, y)
7
8 plt.title("Simple Line Plot")
9 plt.xlabel("X")
10 plt.ylabel("Y")
11 plt.grid(True)
12
13 plt.show()
```

핵심 포인트

`plot()`은 그래프를 만들고, `show()`는 완성된 그래프를 화면에 표시한다. 제목과 축 이름은 그래프가 무엇을 의미하는지 명확하게 전달하기 위해 함께 작성하는 것이 좋다.

3. 그래프 스타일

`plot()`에는 그래프의 모양을 조절하는 여러 옵션을 전달할 수 있다. 자주 사용하는 옵션은 색상, 선 스타일, 마커, 범례 이름이다.

옵션	기능
<code>color</code>	선 색상 지정
<code>linestyle</code>	선 모양 지정
<code>marker</code>	데이터 점 모양 지정
<code>label</code>	범례에 표시할 이름 지정

`label`을 지정한 뒤에는 `plt.legend()`를 호출해야 범례가 화면에 나타난다.

색상은 이름뿐 아니라 RGB 튜플이나 HEX 코드로도 지정할 수 있다. Matplotlib의 RGB 값은 일반적으로 0부터 1 사이의 값을 사용한다.

```
1 color=(1, 0, 0)
2 color="#FF0000"
```

3.1. 최종 코드: 02_plot_style.py

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 4, 5]
4 y = [3, 5, 4, 7, 6]
5
6 plt.plot(
7     x,
8     y,
9     color="red",
10    linestyle="--",
11    marker="o",
12    label="Data"
13 )
14
15 plt.title("Styled Line Plot")
16 plt.xlabel("X")
17 plt.ylabel("Y")
18 plt.grid(True)
19 plt.legend()
20
21 plt.show()
```

핵심 포인트

간단한 그래프는 색상 이름을 사용하는 것이 편리하고, 정확한 색상을 지정해야 할 때는 RGB 튜플이나 HEX 코드를 사용할 수 있다.

4. 여러 개의 선 그래프

하나의 좌표축에서 여러 데이터를 비교하려면 `plt.plot()`을 여러 번 호출한다. `show()`가 호출되기 전까지는 새로 그린 그래프가 같은 좌표축에 계속 추가된다.

각 선에 서로 다른 `label`을 지정하고 `legend()`를 사용하면 어떤 선이 어떤 데이터를 나타내는지 쉽게 구분할 수 있다.

4.1. 최종 코드: 03_multiple_plots.py

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 4, 5]
4
5 student_a = [70, 75, 80, 90, 95]
6 student_b = [65, 72, 78, 85, 90]
7
8 plt.plot(
9     x,
10    student_a,
11    marker="o",
12    label="Student A"
13 )
14
15 plt.plot(
16     x,
17     student_b,
18     marker="s",
19     label="Student B"
20 )
21
22 plt.title("Student Scores")
23 plt.xlabel("Exam")
24 plt.ylabel("Score")
25 plt.grid(True)
26 plt.legend()
27
28 plt.show()
```

핵심 포인트

같은 x축에서 여러 데이터의 변화 추세를 비교할 때는 `plot()`을 반복 호출하고, `label`과 `legend()`를 함께 사용하는 방식이 기본이다.

5. 막대 그래프

막대 그래프는 서로 다른 항목의 크기를 비교할 때 사용한다. `plt.bar(x, y)`에서 `x`에는 항목 이름이나 위치를, `y`에는 각 항목의 값을 전달한다.

`color`로 막대 색상을 지정할 수 있고, `width`로 막대의 너비를 조절할 수 있다. 막대의 높이를 비교할 때는 `grid(axis="y")`를 사용해 가로 방향 격자만 표시하면 읽기 편하다.

5.1. 최종 코드: 04_bar_chart.py

```
1 import matplotlib.pyplot as plt
2
3 fruits = ["Apple", "Banana", "Orange", "Grape"]
```

```

4 sales = [15, 23, 18, 30]
5
6 plt.bar(
7     fruits,
8     sales,
9     color="skyblue",
10    width=0.6
11 )
12
13 plt.title("Fruit Sales")
14 plt.xlabel("Fruit")
15 plt.ylabel("Sales")
16 plt.grid(axis="y")
17
18 plt.show()

```

핵심 포인트

bar()는 이미 항목별 값이 정리된 데이터를 비교하는 그래프이다. 각 막대의 높이가 해당 항목의 값을 나타낸다.

6. 산점도

산점도는 두 변수의 관계를 점으로 나타내는 그래프이다. 데이터 사이에 자연스러운 순서가 없거나 점들을 선으로 연결할 필요가 없을 때 적합하다.

plt.scatter(x, y)를 사용하며, s로 점의 크기를 지정하고 marker로 점의 모양을 바꿀 수 있다.

예를 들어 공부 시간과 시험 점수처럼 두 변수 사이의 관계를 확인할 때 산점도를 사용할 수 있다. 실험 데이터에서는 측정값을 scatter()로 표시하고 이론 곡선을 plot()으로 함께 표시하는 방식도 자주 사용한다.

6.1. 최종 코드: 05_scatter_plot.py

```

1 import matplotlib.pyplot as plt
2
3 height = [160, 165, 170, 175, 180]
4 score = [70, 75, 82, 88, 95]
5
6 plt.scatter(
7     height,
8     score,
9     color="blue",
10    s=80,
11    marker="o"
12 )
13
14 plt.title("Height vs Score")
15 plt.xlabel("Height (cm)")
16 plt.ylabel("Score")
17 plt.grid(True)
18
19 plt.show()

```

핵심 포인트

`scatter()`는 두 변수의 관계나 상관관계를 시각적으로 확인할 때 사용한다. 점 사이를 연결하지 않는다는 점이 기본 선 그래프와 가장 큰 차이이다.

7. 히스토그램

히스토그램은 데이터가 어떤 구간에 얼마나 많이 분포하는지 확인하는 그래프이다. 막대 그래프가 항목별 값을 비교한다면, 히스토그램은 원본 수치 데이터를 여러 구간으로 나눈 뒤 각 구간에 포함되는 데이터 개수를 보여준다.

`plt.hist(data)`를 사용하며, `bins`는 데이터를 몇 개의 구간으로 나눌지를 지정한다. `edgecolor`를 사용하면 각 막대의 경계를 명확하게 표시할 수 있다.

7.1. 최종 코드: 06_histogram.py

```
1 import matplotlib.pyplot as plt
2
3 scores = [
4     72, 75, 81, 83, 84,
5     85, 86, 87, 88, 90,
6     91, 92, 95, 96, 98
7 ]
8
9 plt.hist(
10     scores,
11     bins=5,
12     color="skyblue",
13     edgecolor="black"
14 )
15
16 plt.title("Score Distribution")
17 plt.xlabel("Score")
18 plt.ylabel("Frequency")
19 plt.grid(axis="y")
20
21 plt.show()
```

함수	입력 데이터	목적
<code>bar()</code>	항목별로 이미 정리된 값	항목 간 크기 비교
<code>hist()</code>	개별 원본 수치 데이터	데이터 분포 확인

핵심 포인트

`bins`의 값에 따라 히스토그램의 구간 수가 달라진다. 같은 데이터라도 구간 수를 어떻게 정하느냐에 따라 분포가 다르게 보일 수 있다.

8. 여러 그래프 배치

한 화면에 여러 그래프를 함께 배치하려면 `plt.subplot()`을 사용할 수 있다.

```
1 plt.subplot(rows, columns, index)
```

예를 들어 `plt.subplot(2, 2, 1)`은 화면을 2행 2열로 나눈 뒤 첫 번째 위치를 선택한다. 이후 호출하는 `plot()`, `bar()`, `scatter()`, `hist()`는 현재 선택된 위치에 그려진다.

여러 그래프의 제목이나 축 이름이 겹칠 수 있으므로 마지막에 `plt.tight_layout()`을 사용하면 간격을 자동으로 조정할 수 있다.

8.1. 최종 코드: 07_subplot.py

```

1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 4, 5]
4 y = [2, 4, 6, 8, 10]
5
6 plt.subplot(2, 2, 1)
7 plt.plot(x, y)
8 plt.title("Line")
9
10 plt.subplot(2, 2, 2)
11 plt.bar(x, y)
12 plt.title("Bar")
13
14 plt.subplot(2, 2, 3)
15 plt.scatter(x, y)
16 plt.title("Scatter")
17
18 plt.subplot(2, 2, 4)
19 plt.hist(y)
20 plt.title("Histogram")
21
22 plt.tight_layout()
23 plt.show()

```

핵심 포인트

`subplot()`은 하나의 창을 여러 영역으로 나누는 기능이고, `tight_layout()`은 여러 그래프가 서로 겹치지 않도록 간격을 정리하는 기능이다.

9. Day 9 종합 실습: 학생 데이터 시각화

이번 종합 실습에서는 학생 점수와 공부 시간, 과목별 평균 데이터를 이용해 선 그래프, 막대 그래프, 산점도, 히스토그램을 하나의 화면에 배치한다.

9.1. 프로그램 요구사항

1. 학생 이름과 점수를 이용해 학생별 점수 선 그래프를 그린다.
2. 과목 이름과 평균 점수를 이용해 과목별 평균 막대 그래프를 그린다.
3. 공부 시간과 점수를 이용해 두 변수의 관계를 산점도로 나타낸다.
4. 학생들의 점수 데이터를 이용해 점수 분포 히스토그램을 그린다.
5. `subplot(2, 2, ...)`을 이용해 네 그래프를 한 화면에 배치한다.
6. 각 그래프에 적절한 제목과 축 이름을 작성한다.
7. 필요한 그래프에는 격자를 추가한다.
8. 마지막에 `tight_layout()`과 `show()`를 호출한다.

9.2. 최종 코드: practice_day09.py

```
1 import matplotlib.pyplot as plt
2
3 students = [
4     "Alice", "Bob", "Charlie", "David", "Emma",
5     "Frank", "Grace", "Henry", "Ivy", "Jack"
6 ]
7
8 scores = [72, 75, 81, 83, 84, 85, 86, 87, 88, 90]
9 study_hours = [1, 2, 3, 3, 4, 4, 5, 6, 7, 8]
10
11 subjects = ["Math", "English", "Physics", "Chemistry"]
12 averages = [82, 76, 91, 85]
13
14 plt.subplot(2, 2, 1)
15 plt.plot(
16     students,
17     scores,
18     marker="o"
19 )
20 plt.title("Student Scores")
21 plt.xlabel("Students")
22 plt.ylabel("Scores")
23 plt.grid(True)
24
25 plt.subplot(2, 2, 2)
26 plt.bar(
27     subjects,
28     averages,
29     color="skyblue"
30 )
31 plt.title("Subject Averages")
32 plt.xlabel("Subjects")
33 plt.ylabel("Average Score")
34 plt.grid(axis="y")
35
36 plt.subplot(2, 2, 3)
37 plt.scatter(
38     study_hours,
39     scores,
40     marker="*",
41     s=80
42 )
43 plt.title("Study Hours vs Scores")
44 plt.xlabel("Study Hours")
45 plt.ylabel("Scores")
46 plt.grid(True)
47
48 plt.subplot(2, 2, 4)
49 plt.hist(
50     scores,
51     bins=5,
52     color="green",
53     edgecolor="black"
54 )
55 plt.title("Score Distribution")
56 plt.xlabel("Scores")
57 plt.ylabel("Frequency")
58 plt.grid(axis="y")
```

```

59 plt.tight_layout()
60 plt.show()
61

```

9.3. 프로그램 구조 분석

기능	사용한 개념
학생별 점수 변화 표시	plot()
과목별 평균 비교	bar()
공부 시간과 점수 관계 확인	scatter()
점수 분포 확인	hist()
한 화면에 그래프 배치	subplot()
그래프 간격 자동 조정	tight_layout()
그래프 출력	show()

핵심 포인트

같은 데이터라도 목적에 따라 적절한 그래프가 다르다. 값의 변화는 선 그래프, 항목별 비교는 막대 그래프, 두 변수의 관계는 산점도, 데이터의 분포는 히스토그램이 기본적인 선택이다.

10. 실습 파일 목록

실습

Day 9의 학습 파일은 다음과 같이 관리한다.

- 01_basic_plot.py
- 02_plot_style.py
- 03_multiple_plots.py
- 04_bar_chart.py
- 05_scatter_plot.py
- 06_histogram.py
- 07_subplot.py
- practice_day09.py
- python_day09.tex
- python_day09.pdf

11. 핵심 요약

- Matplotlib는 Python에서 데이터를 그래프로 시각화하기 위한 대표적인 라이브러리이다.
- matplotlib.pyplot은 일반적으로 plt라는 별칭으로 불려온다.
- plot(x, y)은 데이터 점을 선으로 연결한 선 그래프를 만든다.
- title(), xlabel(), ylabel()로 제목과 축 이름을 지정한다.

- `grid()`는 그래프에 격자를 표시한다.
- `color`, `linestyle`, `marker`로 그래프 스타일을 조절할 수 있다.
- 색상은 이름, RGB 튜플, HEX 코드 등으로 지정할 수 있다.
- `label`과 `legend()`를 사용하면 여러 그래프를 쉽게 구분할 수 있다.
- 하나의 좌표축에 여러 선을 그리려면 `plot()`을 여러 번 호출한다.
- `bar()`는 서로 다른 항목의 크기를 비교할 때 사용한다.
- `scatter()`는 두 변수의 관계를 점으로 표현한다.
- `s`는 산점도의 점 크기를 조절한다.
- `hist()`는 원본 데이터의 분포를 여러 구간으로 나누어 표시한다.
- `bins`는 히스토그램의 구간 수를 지정한다.
- `subplot()`은 한 화면을 여러 영역으로 나누어 그래프를 배치한다.
- `tight_layout()`은 여러 그래프 사이의 간격을 자동으로 정리한다.
- `show()`는 완성된 그래프를 화면에 표시한다.
- Matplotlib의 고급 기능은 실제 프로젝트에서 필요한 시점에 추가로 학습한다.